

Natural Computing 2023

First Exam* — July 17th, 2023

Please mind the following:

- Fill in your **personal information** in the fields below.
- You may use an unmarked dictionary during the exam. **No** additional tools (like calculators, e.g.) might be used.
- Fill in all your answers **directly into this exam sheet**. You may use the backside of the individual sheets or ask for additional paper. In any case, make sure to mark **clearly** which question you are answering. Do not use pens with green or red color or pencils for writing your answers. You may give your answers in both **German and English**.
- Points annotated for the individual tasks only serve as a preliminary guideline.
- At the end of the exam hand in your **whole exam sheet**. Please make sure you do not undo the binder clip!
- To forfeit your exam (“entwerten”), please cross out clearly this **cover page** as well as **all pages** of the exam sheet. This way the exam will not be evaluated and will not be counted as an exam attempt.
- Please place your LMU-Card or student ID as well as photo identification (“Lichtbildausweis”) clearly visible on the table next to you. Note that we need to check your data with the data you enter on this cover sheet.

Time Limit: 90 minutes

First Name:																											
Last Name:																											
Matriculation Number:																											
<table border="1"><tr><td>Topic 1</td><td>max. 10 pts.</td><td>pts.</td></tr><tr><td>Topic 2</td><td>max. 15 pts.</td><td>pts.</td></tr><tr><td>Topic 3</td><td>max. 14 pts.</td><td>pts.</td></tr><tr><td>Topic 4</td><td>max. 12 pts.</td><td>pts.</td></tr><tr><td>Topic 5</td><td>max. 15 pts.</td><td>pts.</td></tr><tr><td>Topic 6</td><td>max. 12 pts.</td><td>pts.</td></tr><tr><td>Topic 7</td><td>max. 12 pts.</td><td>pts.</td></tr><tr><td colspan="2">Total max. 90 pts.</td><td>pts.</td></tr></table>				Topic 1	max. 10 pts.	pts.	Topic 2	max. 15 pts.	pts.	Topic 3	max. 14 pts.	pts.	Topic 4	max. 12 pts.	pts.	Topic 5	max. 15 pts.	pts.	Topic 6	max. 12 pts.	pts.	Topic 7	max. 12 pts.	pts.	Total max. 90 pts.		pts.
Topic 1	max. 10 pts.	pts.																									
Topic 2	max. 15 pts.	pts.																									
Topic 3	max. 14 pts.	pts.																									
Topic 4	max. 12 pts.	pts.																									
Topic 5	max. 15 pts.	pts.																									
Topic 6	max. 12 pts.	pts.																									
Topic 7	max. 12 pts.	pts.																									
Total max. 90 pts.		pts.																									

*Corrections to the original handout are marked in red in this file.

1 General Knowledge / Multiple Choice

10pts

For each of the following questions select **one** correct answer ('1 of n '). Every correct answer is awarded one point. Multiple answers or incorrect answers will be marked with zero points.

(a) Which of the following disciplines is not a common inspiration for natural computing? (1pt)

i Physics	ii Biology	iii <u>Architecture</u>	iv Chemistry
-----------	------------	-------------------------	--------------

(b) In Conway's game of life, a pattern that emits other patterns (most notably space ships) is called...? (1pt)

i sword	ii spear	iii <u>gun</u>	iv tank
---------	----------	----------------	---------

(c) According to Stephen Wolfram we can classify 1D cellular automata into roughly four (non-exclusive) groups. Which of the following is not part of Wolfram's classification system? (1pt)

i <u>wave form</u>	ii uniformity	iii random	iv repetition
--------------------	---------------	------------	---------------

(d) Erwin Schrödinger imagined a thought experiment where a cat's survival in a sealed box depends on a quantum measurement. Which is not a state that quantum physics might consider the cat to be in at any point in time? (1pt)

i $ dead\rangle$	ii $ alive\rangle$	iii <u>$dead\rangle \otimes alive\rangle$</u>	iv $\frac{1}{\sqrt{2}} dead\rangle + \frac{1}{\sqrt{2}} alive\rangle$
------------------	--------------------	--	---

(e) An important algorithm for optimization using quantum effects that is usually implemented in specialized hardware is called...? (1pt)

i quantum appearing	ii quantum ascending	iii quantum advantage	iv <u>quantum annealing</u>
---------------------	----------------------	-----------------------	-----------------------------

(f) In accordance with its physical inspiration, the chance that simulated annealing accepts a worse solution candidate as its next tends to...? (1pt)

i <u>decrease over time</u>	ii stay the same	iii increase over time	iv be way too low in general
-----------------------------	------------------	------------------------	------------------------------

(g) The No Free Lunch theorem states that — when averaged over all possible problems — all optimization algorithms perform...? (1pt)

i worse than random search	ii <u>as good as random search</u>	iii better than random search	iv the best when used together
----------------------------	------------------------------------	-------------------------------	--------------------------------

(h) What are the two goals that all metaheuristic optimization algorithms need to balance? (1pt)

i experience/ expression	ii exponential/ extrapolation	iii expectation/ exasperation	iv <u>exploration/ exploitation</u>
--------------------------	-------------------------------	-------------------------------	-------------------------------------

(i) According to Richard Dawkins there are three central properties of a good self-replicator. **Which is not one of those?** (1pt)

i fecundity	ii longevity	iii <u>clarity</u>	iv copy-fidelity
-------------	--------------	--------------------	------------------

(j) The self-replicator conjectured by some to be responsible for “the evolution of culture” is called...? (1pt)

i scene	ii <u>meme</u>	iii theme	iv ✓✓seen
---------	----------------	-----------	-----------

2 Game of Life

15pts

The following definition was given in the lecture.

Definition 1 (Conway's game of life (standard)). Let $G = (V, E)$ be a graph with vertices V and (undirected) edges $E \subseteq V \times V$ with a fixed degree of 8 for all nodes. We define $surroundings : V \rightarrow \wp(V)$ via

$$surroundings(v) = \{w \mid (v, w) \in E\}$$

so that $|surroundings(v)| = 8$ and $v \notin surroundings(v)$ for all $v \in V$. A state $x \in \mathcal{X}$ is a mapping of vertices to the labels $\{dead, alive\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{dead, alive\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$$|v|_{x_t} = |\{w \mid w \in surroundings(v) \wedge x_t(w) = alive\}|.$$

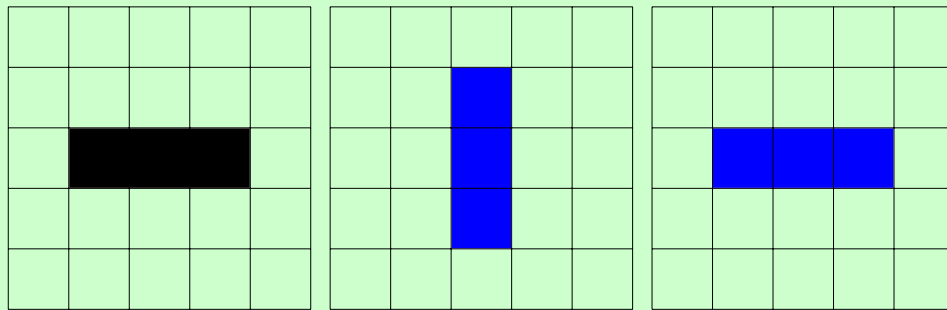
In the game of life, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via

$$x_{t+1}(v) = \begin{cases} dead & \text{if } |v|_{x_t} \leq 1, \\ x_t(v) & \text{if } |v|_{x_t} = 2, \\ alive & \text{if } |v|_{x_t} = 3, \\ dead & \text{if } |v|_{x_t} \geq 4, \end{cases}$$

for all $v \in V$. A tuple (G, x_0) is called an instance of the game of life for initial state $x_0 \in \mathcal{X}$.

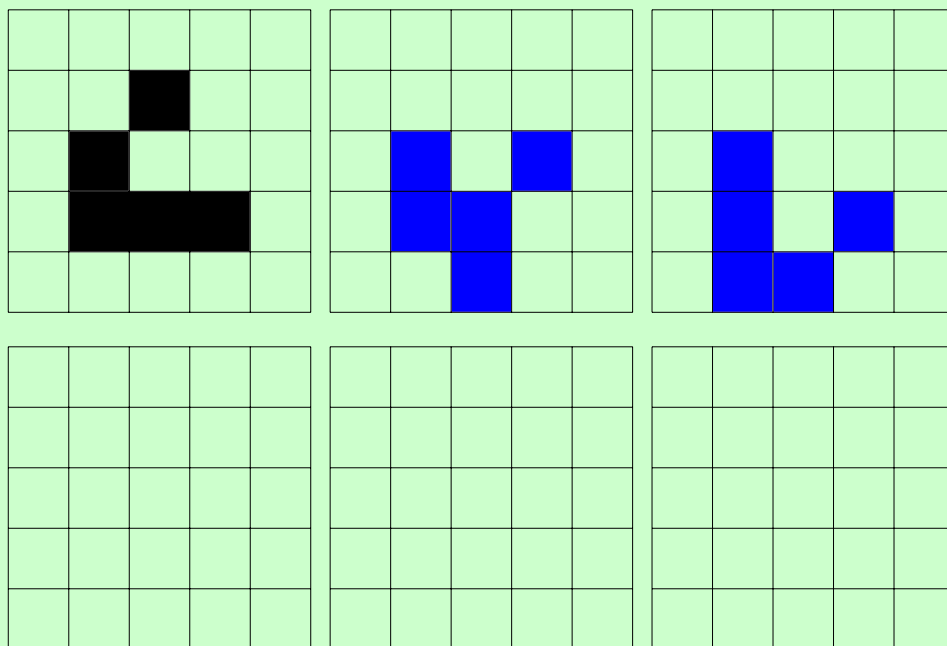
For this task, we play Conway's game of life on a 5×5 grid with a Moore neighborhood (i.e., every node is connected via edges to its 8 neighbors in the grid) featuring wrap-around at the borders. In the lecture, we discussed various types of patterns including still lives, oscillators, and space ships.

- (a) **Evolve the pattern below in the game of life for as many steps as necessary to show that it is an oscillator. Give your reasoning.** You may use the empty grids to the right of the pattern to draw its subsequent states. (4pts)



The configuration repeats, so it is an oscillator.

- (b) **Evolve the pattern below in the game of life for as many steps as necessary to show that it is definitely not an oscillator. Give your reasoning.** You may use the empty grids to the right and below of the pattern to draw its subsequent states. (7pts)



The pattern repeats rotated and shifted, so the trend observed so far continues and we can never achieve the original pattern again. (Of course, you can also fill in two more fields and see that the pattern repeats shifted.)

(c) Give a formal definition of an oscillator by completing Definition 2 below in an appropriate way. (4pts)

Definition 2 (oscillator in Conway's game of life). Let $\langle x_0, x_1, \dots \rangle$ be an infinite sequence of states in the game of life so that x_{t+1} is derived from x_t as shown in Definition 1. x_0 is called an *oscillator* iff ...

... $\exists t \in \mathbb{N} : x_t = x_0$

3 Cellular Automata

14pts

The following definitions were given in the lecture.

Definition 3 (cellular automaton). Let $G = (V, E)$ be a graph with vertices V and edges $E \subseteq V \times V$. Let $neighborhood : V \rightarrow V^{d+1}$ for some neighborhood degree $d \in \mathbb{N}$ be a function that returns an ordered vector of neighbors of a given node v , always including v itself. A state $x \in \mathcal{X}$ is a mapping of vertices to the values $\{0, 1\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{0, 1\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. The evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via a function $f : \{0, 1\}^{d+1} \rightarrow \{0, 1\}$ so that

$$x_{t+1}(v) = f(x_t(u_1), \dots, x_t(u_{d+1}))$$

where $\vec{u} = \langle u_1, \dots, u_{d+1} \rangle = neighborhood(v)$.

A tuple (G, f, x_0) is called a cellular automaton with initial state $x_0 \in \mathcal{X}$.

Definition 4 (1D cellular automaton). A cellular automaton (G, f, x_0) is called a 1D cellular automaton iff all vertices have exactly one incoming edge (“left neighbor”) and one outgoing edge (“right neighbor”) and $neighbors(v) = \langle u, v, w \rangle$ where $(u, v) \in E$ and $(v, w) \in E$ and thus $d = 2$. Subsequently, we usually write

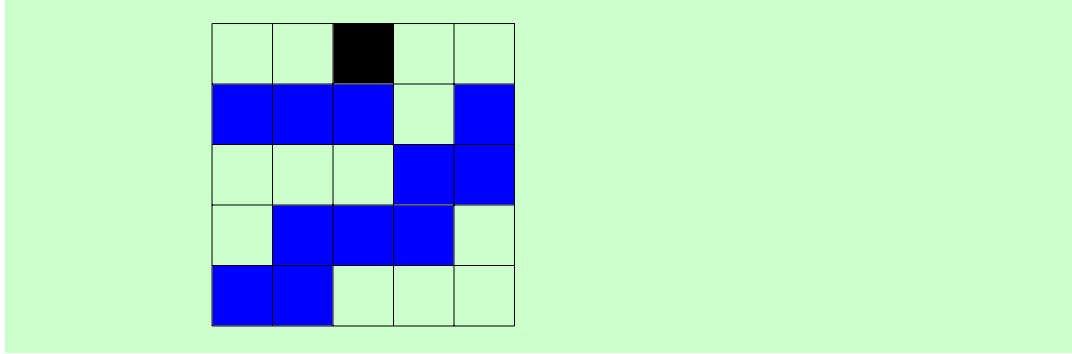
$$x_{t+1}(v) = f(x_t(u), x_t(v), x_t(w))$$

for the evolution where u is the left neighbor of v and w is the right neighbor of v .

A 1D cellular automaton is often given via a *rule* $n \in [0; 255] \subset \mathbb{N}$ where each digit in n ’s binary representation encodes one output of f so that the digit for the base value 128 encodes the output of $f(1, 1, 1)$, 64 for $f(1, 1, 0)$, and so on sorted by decreasing numerical value.

(a) **Fill in the graphical template for a 1D cellular automaton’s function f given the rule $n = 47$.** (2pts)

(b) With the function f for rule 47 as given in task (a), evolve this initial configuration by hand for four generations. Use one row per generation. Note that the nodes of one state form a ring, i.e., they wrap around at the borders. (6pts)



(c) We now introduce a *lingering cellular automaton* as given in Definition 5 below.

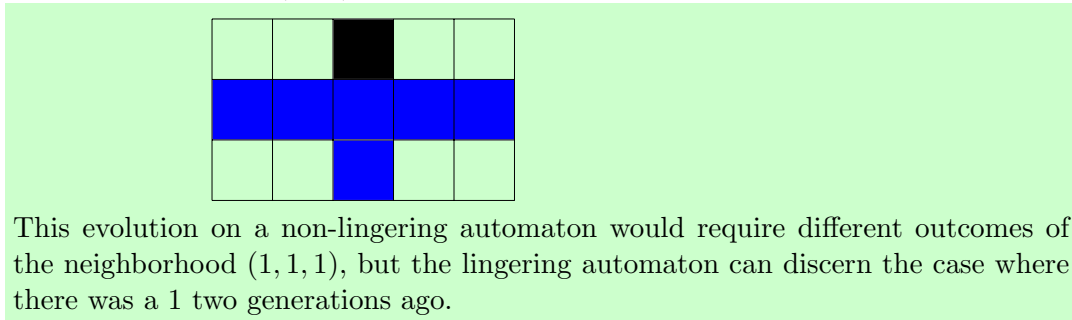
Definition 5 (lingering cellular automaton). A lingering cellular automaton is defined exactly like a cellular automaton (cf. Definition 3) except that $f : \{0, 1\}^{d+2} \rightarrow \{0, 1\}$ now receives one additional input so that

$$x_{t+1}(v) = f(x_t(u_1), \dots, x_t(u_{d+1}), \mathbf{h})$$

where $\vec{u} = \langle u_1, \dots, u_{d+1} \rangle = \text{neighborhood}(v)$

$$\text{and } \mathbf{h} = \begin{cases} 0 & \text{if } t = 0, \\ x_{t-1}(v) & \text{otherwise.} \end{cases}$$

From the initial configuration shown below, give an evolution that can be generated by a lingering 1D cellular automaton, but cannot be generated by a standard (non-lingering) 1D cellular automaton. Use one row per generation. Write down the reason your evolution can only originate from a lingering cellular automaton. Note that the nodes of one state form a ring, i.e., they wrap around at the borders. (6pts)



4 Quantum Computing

12pts

The following definitions were given in the lecture.

Definition 6 (quantum state). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{Q}(\mathcal{X})$ be the space of superposition states with bases in $\mathcal{X}$, i.e., each $q \in \mathcal{Q}(\mathcal{X})$ has the form $q = \sum_{x \in \mathcal{X}} c_x |x\rangle$ where $c_x \in \mathbb{C}$ for any $x \in \mathcal{X}$ is called an amplitude so that $\sum_{x \in \mathcal{X}} |c_x|^2 = 1$ and $|x\rangle$ (read “ket x ”) denotes a vector corresponding to the state $x \in \mathcal{X}$.

We use the following terminology:

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.
- If $\mathcal{X} = \mathcal{X}_A \times \mathcal{X}_B$, i.e., $\mathcal{X}$ can be constructed as a product of two other state spaces $\mathcal{X}_A, \mathcal{X}_B$, then it follows that q has the form $q = \sum_{\substack{x_A \in \mathcal{X}_A, \\ x_B \in \mathcal{X}_B}} c_{x_A, x_B} |x_A\rangle \otimes |x_B\rangle$ where $\otimes$ is the tensor product. If q cannot be written as $q = (\sum_{x_A \in \mathcal{X}_A} c_{x_A} |x_A\rangle) \otimes (\sum_{x_B \in \mathcal{X}_B} c_{x_B} |x_B\rangle)$, then we call q an *entangled* state.
- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as $q = c_0 |0\rangle + c_1 |1\rangle$ for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.
- If $\mathcal{X} = \{0, 1\}^n$ for any $n \in \mathbb{N}$ (i.e., any $x \in \mathcal{X}$ is a bit register), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit register* of size n and can be written as $q = \sum_{i=0}^{2^n-1} c_i |i\rangle = c_{0\dots 0} |0\dots 0\rangle + \dots + c_{1\dots 1} |1\dots 1\rangle$ where we usually denote i in binary as illustrated above.
- When we *measure* a quantum state $q \in \mathcal{Q}(\mathcal{X})$, we generate classical state $M(q) \in \mathcal{X}$ so that for all $x \in \mathcal{X}$ it holds that $\Pr(M(q) = x) = |c_x|^2$.

Definition 7 (QUBO). Let $Q \in \mathbb{R}^{n \times n}$ be an arbitrary matrix called QUBO matrix. The process of finding a vector $\vec{x} \in \mathbb{B}^n$ so to minimize $H(\vec{x}) = x^\top Q x = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} Q_{ij} x_i x_j$ is called *quadratic unconstrained binary optimization (QUBO)*. W.l.o.g. we can assume that Q is a (non-strictly) upper triangular matrix.

(a) **Show why** $|a\rangle = 0.98 \cdot |00\rangle + 0.01 \cdot |01\rangle + 0.01 \cdot |10\rangle$ **is not a valid quantum state.** (4pts)

$$\begin{aligned} 0.98 + 0.01 + 0.01 &= 1 \\ \text{but } |0.98|^2 &< 0.98 \text{ and } |0.01|^2 < 0.01 \\ \text{so } |0.98|^2 + |0.01|^2 + |0.01|^2 &< 1 \end{aligned}$$

(Of course, you can also just compute all values.)

(b) **Show why** $|b\rangle = \frac{1}{\sqrt{4}} \cdot |00\rangle + \frac{1}{\sqrt{4}} \cdot |01\rangle + \frac{1}{\sqrt{4}} \cdot |10\rangle + \frac{1}{\sqrt{4}} \cdot |11\rangle$ **is not an entangled quantum state.** (4pts)

It can be written as

$$\left(\frac{1}{\sqrt{2}} \cdot |0\rangle + \frac{1}{\sqrt{2}} \cdot |1\rangle \right) \otimes \left(\frac{1}{\sqrt{2}} \cdot |0\rangle + \frac{1}{\sqrt{2}} \cdot |1\rangle \right)$$

(c) Consider the QUBO matrix Q given via $Q = \begin{pmatrix} -1 & 2 \\ & 3 \end{pmatrix}$.

For the solution candidate $\vec{x} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ **compute the energy** $H(\vec{x})$. This is not the optimal energy for Q , however. **Give the solution candidate** $\vec{x}^*$ **so that** $H(\vec{x}^*)$ **is optimal. What is the value of** $H(\vec{x}^*)$? (4pts)

$$\begin{aligned} H(\vec{x}) &= \begin{pmatrix} 0 & 1 \end{pmatrix} \begin{pmatrix} -1 & 2 \\ & 3 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = 3 \\ \vec{x}^* &= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \text{ with } H(\vec{x}^*) = -1 \end{aligned}$$

5 Optimization

15pts

We give a shortened version of the definition for optimization processes from the lecture and then introduce a new optimization algorithm called *discrete hill climbing*.

Definition 8 (optimization (shortened)). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization.

An optimization run of length $g+1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t , $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{\text{def}} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

Algorithm 1 (discrete hill climbing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. We assume $\mathcal{X} = \mathbb{Z}$ and $\mathcal{T} = \mathbb{Z}$. The process $\mathcal{D}$ continues via discrete hill climbing iff e is given via

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t - 1, x_t, x_t + 1\}} \tau(x).$$

Note that the subscript above reads “ $x \in \{x_t - 1, x_t, x_t + 1\}$ ”, i.e., 1 is subtracted and added to x_t , not to t .

(a) Let the target function to be minimized be

$$\tau(x) = \min\{(x-2)^2, (x-5)^2 + 1\}.$$

Let the initial state be $x_0 = 7$. **Execute discrete hill climbing (cf. Algorithm 1) in this setting for 3 time steps, i.e., compute x_1, x_2, x_3 . Does discrete hill climbing reach the global optimum of τ within these time steps? Where is the global optimum x^* of τ and what is $\tau(x^*)$?** (8pts)

step 1:

$$\begin{aligned}\tau(x_0 - 1) &= \tau(6) = \min\{16, 2\} = 2 \\ \tau(x_0) &= \tau(7) = \min\{25, 5\} = 5 \\ \tau(x_0 + 1) &= \tau(8) = \min\{36, 10\} = 10 \\ \implies x_1 &= 6\end{aligned}$$

step 2:

$$\begin{aligned}\tau(x_1 - 1) &= \tau(5) = \min\{9, 1\} = 1 \\ \tau(x_1) &= \tau(6) = 2 \\ \tau(x_1 + 1) &= \tau(7) = 5 \\ \implies x_2 &= 5\end{aligned}$$

step 3:

$$\begin{aligned}\tau(x_2 - 1) &= \tau(4) = \min\{4, 2\} = 2 \\ \tau(x_2) &= \tau(5) = 1 \\ \tau(x_2 + 1) &= \tau(6) = 2 \\ \implies x_3 &= 5\end{aligned}$$

Discrete hill climbing does not reach the global optimum of $x^* = 2$ with $\tau(x^*) = 0$.

(b) Consider a new algorithm defined below...

Algorithm 2 (overly enthusiastic discrete hill climbing (OEDHC)). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. We assume $\mathcal{X} = \mathbb{Z}$ and $\mathcal{T} = \mathbb{Z}$. The process $\mathcal{D}$ continues via overly enthusiastic discrete hill climbing (OEDHC) iff e is given via

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} x_t - 5 & \text{if } \tau(x_t - 1) < \tau(x_t) \text{ and } \tau(x_t - 1) < \tau(x_t + 1), \\ x_t + 5 & \text{if } \tau(x_t + 1) < \tau(x_t) \text{ and } \tau(x_t + 1) < \tau(x_t - 1), \\ x_t & \text{otherwise.} \end{cases}$$

Let the target function to be minimized be $\tau(x) = \min\{(x - 2)^2, (x - 5)^2 + 1\}$ and the initial state be $x_0 = 7$ as in the task above. **Execute OEDHC (cf. Algorithm 2) in this setting for one time step, i.e., compute x_1 . Does overly enthusiastic discrete hill climbing (OEDHC) reach the global optimum of τ within this time step?** (3pts)

step 1:

$$\tau(x_0 - 1) = \tau(6) = 2$$

$$\tau(x_0) = \tau(7) = 5$$

$$\tau(x_0 + 1) = \tau(8) = 10$$

$$\implies x_1 = 2$$

Yes, OEDHC does reach the global optimum of $x^* = 2$ with $\tau(x^*) = 0$.

(c) **Give a new target function $\tau' : \mathbb{R} \rightarrow \mathbb{R}$ for which OEDHC never reaches the global optimum x^* of τ' for all $x_0 \neq x^*$. Briefly explain why. Is OEDHC a good idea for a search algorithm? Briefly explain why.** (4pts)

$$\tau'(x) = \begin{cases} 0 & \text{if } x = x^*, \\ 1 & \text{otherwise.} \end{cases}$$

In this setting, OEDHC will just never move. If it does not start at $x_0 = x^*$ by chance, it will never reach the global optimum.

OEDHC is not a great algorithm because we do not actually have any idea about $x_t \pm 1$ when jumping to it.

6 Artificial Chemistries / Soups

12pts

Recall the following definition from the lecture:

Definition 9 (artificial chemistry, soup). Let $\mathcal{X}$ be a state space. Let $\mathcal{R}$ be a set of reaction rules $R \in \mathcal{R}$ where each R is a function $R : \mathcal{X}_R^{\text{dom}} \rightarrow \mathcal{X}_R^{\text{cod}}$ for $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$. Let $\mathcal{A}(X, \mathcal{R})$ be the set of all *reaction materials* from $X \in \wp^*(\mathcal{X})$ for all rules in $\mathcal{R}$, i.e.,

$$\mathcal{A}(X, \mathcal{R}) = \{(X', R) \mid X' \subseteq X \wedge X' \in \mathcal{X}_R^{\text{dom}} \wedge R \in \mathcal{R}\}.$$

Let $A : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X}) \times \mathcal{R}$ be a possibly randomized or non-deterministic function that takes a multi-set $X \in \wp^*(\mathcal{X})$ and returns a multi-set of reactants and a reaction rule to perform so that $A(X) \in \mathcal{A}(X, \mathcal{R})$. The tuple $(\mathcal{X}, \mathcal{R}, A)$ defines an artificial chemistry. A multi-set $X \in \wp^*(\mathcal{X})$ of an artificial chemistry is also called soup; each element $x \in X$ is also called a particle. The evolution of a soup for g generations is a sequence $\langle X_0, \dots, X_g \rangle$ so that X_0 is given as the initial soup and, for all t with $0 \leq t \leq g-1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where $(X', R) = A(X_t)$.

Reaction rules are usually written in the form $R = \sum_i x_i \rightarrow \sum_j y_j$ for $x_i, y_j \in \mathcal{X}$, $i, j \in \mathbb{N}$, $\{x_i\}_i \in \mathcal{X}_R^{\text{dom}}$, $\{y_j\}_j \in \mathcal{X}_R^{\text{cod}}$, and $R(\{x_i\}_i) = \{y_j\}_j$.

For the following tasks we define an artificial chemistry $\mathcal{C} = (\mathcal{X}, \mathcal{R}, A)$ where $\mathcal{X} = \{\text{COCONUT}, \text{TREE}, \text{ORCA}, \text{HORSE}, \text{SHORE}\}$, $\mathcal{R} = \{R_1, \dots, R_7\}$ with

$$\begin{aligned} R_1 &= \text{TREE} \rightarrow \text{TREE} + \text{COCONUT}, \\ R_2 &= \text{COCONUT} + \text{SHORE} \rightarrow \text{TREE} + \text{SHORE}, \\ R_3 &= \text{COCONUT} + \text{COCONUT} \rightarrow \text{HORSE}, \\ R_4 &= \text{ORCA} + \text{COCONUT} \rightarrow \text{ORCA}, \\ R_5 &= \text{ORCA} + \text{ORCA} \rightarrow \text{ORCA}, \\ R_6 &= \text{ORCA} + \text{HORSE} \rightarrow \text{ORCA}, \\ R_7 &= \text{ORCA} + \text{HORSE} + \text{HORSE} \rightarrow \text{HORSE}, \end{aligned}$$

and $A(X) \sim \mathcal{A}(X, \mathcal{R})$.

(a) Give a possible evolution of the soup

$$X_0 = \{\text{COCONUT}, \text{SHORE}, \text{ORCA}\}$$

for as many time steps as necessary until a HORSE appears. (Hint: Instead of random choices, you can decide which rules to apply to make the process quicker. Use the table below.) (4pts)

time step	chosen rule	current soup
0	none	$\{\text{COCONUT}, \text{SHORE}, \text{ORCA}\}$
1	R_2	$\{\text{TREE}, \text{SHORE}, \text{ORCA}\}$
2	R_1	$\{\text{TREE}, \text{COCONUT}, \text{SHORE}, \text{ORCA}\}$
3	R_1	$\{\text{TREE}, \text{COCONUT}, \text{COCONUT}, \text{SHORE}, \text{ORCA}\}$
4	R_3	$\{\text{TREE}, \text{HORSE}, \text{SHORE}, \text{ORCA}\}$
5		

(b) Let $X = \{\text{ORCA}, \text{ORCA}, \text{HORSE}, \text{HORSE}\}$. What is $\mathcal{A}(X)$? What are all possible final soups if we keep running reactions on X for as long as we can? (5pts)

$$\mathcal{A}(X) = \{(\{\text{ORCA}, \text{ORCA}\}, R_5),$$

$$(\{\text{ORCA}, \text{HORSE}\}, R_6),$$

$$(\{\text{ORCA}, \text{HORSE}, \text{HORSE}\}, R_7)\}$$

We will end up with the soup $\{\text{ORCA}\}$ or $\{\text{HORSE}\}$.

(c) Now let $X = X_{\text{Orcas}} \uplus \{x\}$ where X_{Orcas} is a multi-set consisting of 100 ORCA particles and $x \in \mathcal{X}$ is one single mystery particle. We now run studies on the soup X for an extremely long time and every time we do so, we observe that in late soups there are no ORCA particles anymore. What is the only plausible mystery particle x ? State your reasoning. (3pts)

$x = \text{TREE}$ because trees cannot die out and keep on producing coconuts, which keep on producing Horses, which eventually have a chance of killing Orcas.

7 Evolutionary Computing

12pts

If necessary, you can recollect the definition for evolutionary algorithms as used in the lecture on this page. (Hint: Try the tasks first before reading this lengthy definition again.)

Algorithm 3 (typical evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process.

- Let $\sigma_N^{survivors}, \sigma_N^{parents} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be (possibly randomized or non-deterministic) selection functions that return $N \in \mathbb{N}$ individuals, i.e., $|\sigma_N(X)| = N$ and $\sigma_N(X) \subseteq X$ for all X . They may possibly depend on $\mathcal{E}$ (most commonly ϕ). Let $\varrho_N^{mutants} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be a special selection function that, given a population X , returns a random subset $X' \subseteq X$ so that $|X'| = N$. Note that $\varrho_{|X|}(X) = X$ for all X .
- Let $mutate : \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) single-individual mutation function. We write $mutate[_] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the per-individual application of $mutate$, i.e., $mutate[X] = \{mutate(x) : x \in X\}$.
- Let $recombine : \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) two-parent recombination function. We write $recombine[_] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the random-pairing application of $recombine$, i.e., $recombine[X] = \{recombine(x_1, x_2)\} \uplus recombine[X \setminus \{x_1, x_2\}]$ with $x_1, x_2 \sim X$ and $recombine[\{x\}] = \emptyset$ for all x as well as $recombine[\emptyset] = \emptyset$.
- Let $migrate : \emptyset \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) individual creation function. We write $migrate_N : \emptyset \rightarrow \wp^*(\mathcal{X})$ with $N \in \mathbb{N}$ for the iterated application of $migrate$, i.e., $migrate_N[] = \{migrate()\} \uplus migrate_{N-1}[]$ with $migrate_0[] = \emptyset$.

The process $\mathcal{E}$ continues via a typical evolutionary algorithm if E has the form

$$E(\langle X_u \rangle_{0 \leq u \leq t}, \phi) = X_{t+1} = \sigma_{|X_t|}^{survivors}(X_t \uplus mutate[\varrho_M^{mutants}(X_t)] \uplus recombine[\sigma_{2C}^{parents}(X_t)] \uplus migrate_H[])$$

where $M \in \mathbb{N}$ is the number of mutants produced per generation, $C \in \mathbb{N}$ is the number of children produced per generation, and $H \in \mathbb{N}$ is the number of migrants (sometimes called hypermutants) generated per generation. Especially M is often expressed as a rate for random choice within the population, i.e., $M = \lceil m \cdot |X_t| \rceil$ where $m \in \mathbb{R}$ is called mutation rate.

We now consider an evolutionary algorithm that we use to program a swallow bot, i.e., a robot of a small bird that we might use to carry coconuts, for example. For a little bit of biological plausibility, we encode the swallow bot's behavior via strings made up from 3000 letters A, C, G , and T , i.e., $\mathcal{X} = \{A, C, G, T\}^{3000}$.

(a) A nice one-point crossover function should recombine exactly one continuous part from each parent into one child. Naturally, all genes need to be at the correct place still. Furthermore, we assume that the recombination function receives the parents ordered by fitness, but any parent should have an equal chance of passing on any gene. **In a suitable formalism (Python, pseudo-code, mathematics, ...), give a nice one-point crossover function for the state space $\mathcal{X}$ as defined above.** (6pts)

Note that you can use the Python function `random.random()`, which returns a random floating point number between 0.0 (inclusive) and 1.0 (exclusive), i.e., $[0.0, 1.0)$, and you can use the notation $x \sim X$ to sample from a finite set X .

$$\begin{aligned} \text{recombine}(x_1, x_2) &= x_a[:i] \uplus x_b[i:] \\ \text{where } (x_a, x_b) &\sim \{(x_1, x_2), (x_2, x_1)\} \\ \text{and } i &\sim [1; 2999] \subset \mathbb{N}. \end{aligned}$$

Alternative solution:

```
def recombine(x_1, x_2):
    let parent_a ~ {x_1, x_2}
    let parent_b ~ {x_1, x_2} \ {parent_a}
    let i = floor(random.random() * 2999) + 1
    return parent_a[:i] \uplus parent_b[i:]
```

(b) We want our swallow bot to both have a high velocity and a high carrying capacity. Our swallow bot simulation provides us with both properties via two separate target functions

$$\begin{aligned}\tau_c &: \mathcal{X} \rightarrow \mathbb{R} \\ \text{and } \tau_v &: \mathcal{X} \rightarrow \mathbb{R}\end{aligned}$$

so that, for a swallow bot with code $x \in \mathcal{X}$, $\tau_c(x)$ denotes its carrying capacity and $\tau_v(x)$ denotes its top velocity (when unladen). We have no way to meaningfully compare the values of both target functions with one another, but we can naturally compare values given by the same target function. Note that both τ_c and τ_v are to be maximized.

We now require a beautiful *multi-objective* selection function $\sigma_N^* : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$, which always returns exactly $N \in \mathbb{N}$ individuals from its input population X with $|X| \gg N$ and makes sure that both individuals with good τ_c and τ_v are potentially included. Furthermore, an individual that has the best target value in the population for both target functions at the same time should be guaranteed to be selected. **In a suitable formalism (Python, pseudo-code, mathematics, ...), give such a beautiful multi-objective selection function.** (6pts)

Hint: You may want to build upon cut-off selection or tournament selection.

$$\sigma_0^*(X) = \emptyset$$

$$\sigma_N^*(X) = \sigma_{N-1}^*(X \setminus \{x_1, x_2\}) \uplus \begin{cases} \{x_1\} & \text{if } \tau_c(x_1) > \tau_c(x_2) \wedge \tau_v(x_1) > \tau_v(x_2) \\ \{x_2\} & \text{if } \tau_c(x_2) > \tau_c(x_1) \wedge \tau_v(x_2) > \tau_v(x_1) \\ \{x_R\} & \text{otherwise} \end{cases}$$

where $x_1, x_2 \sim X$ and $x_R \sim \{x_1, x_2\}$

Alternative solution:

```
def select*(N, xs):
    let selected_xs = {}
    while len(selected_xs) < N:
        let x_1 ~ xs
        let x_2 ~ xs
        let x_R ~ {x_1, x_2}
        if tau_c(x_1) > tau_c(x_2) and tau_v(x_1) > tau_v(x_2):
            selected_xs.add(x_1)
        elif tau_c(x_2) > tau_c(x_1) and tau_v(x_2) > tau_v(x_1):
            selected_xs.add(x_2)
        else:
            selected_xs.add(x_R)
        xs.remove(x_1)
        xs.remove(x_2)
    return selected_xs
```

Appendix.

On this page, you find additional grid fields if you need them for the tasks in this exam. Mark clearly on the task's page, if the solution to a task is to be found here, and mark clearly on this page here, which task a solution belongs to.

